Introduction à ReactJS, Architecture et Outillage Moderne

Objectifs Pédagogiques

À l'issue de cette session, vous serez capables de :

1. Expliquer les raisons historiques et techniques qui ont conduit à la création de ReactJS.
2. Différencier l'approche impérative (manipulation directe du DOM) de l'approche déclarative (l'UI comme fonction directe de l'état).
3. Décrire le fonctionnement interne du DOM Virtuel, de l'algorithme de Réconciliation et du moteur React Fiber.
4. Maîtriser l'écosystème d'exécution local autour de Node.js et npm (rôle de package.json, SemVer, scripts).
5. Comprendre et configurer la chaîne de compilation d'une Single Page Application avec Webpack et Vite.

---

1. Historique, Genèse et Philosophie de ReactJS

1.1 Des Applications Multi-Pages (MPA) aux Single Page Applications (SPA)

Historiquement, le Web reposait sur le modèle **MPA (Multi-Page Application)** : chaque action de l'utilisateur (clic sur un lien, soumission d'un formulaire) déclenchait une requête HTTP vers le serveur, qui générait une nouvelle page HTML complète et la renvoyait au navigateur. Ce cycle entraînait un rechargement visuel complet, une perte du contexte d'exécution JavaScript et une latence perceptible.

Avec la montée en puissance des moteurs JavaScript (notamment V8 en 2008) et la standardisation d'AJAX (XMLHttpRequest, puis l'API fetch), le paradigme **SPA (Single Page Application)** s'est imposé :

- Le navigateur charge **un unique document HTML** initial.
- Les interactions ultérieures ne rechargent plus la page : elles échangent uniquement des données (généralement au format JSON) avec des API REST ou GraphQL.
- Le moteur JavaScript met à jour dynamiquement l'interface directement dans le navigateur.

Étape du cycle	Architecture MPA Classique	Architecture SPA Moderne (React)
Chargement initial	Le serveur génère et renvoie la page HTML complète.	Le serveur renvoie un fichier index.html minimal et le bundle JavaScript.
Navigation / Action	Nouvelle requête HTTP complète vers le serveur.	Appel asynchrone (fetch) vers une API retournant uniquement du JSON.
Mise à jour de l'écran	Destruction et rechargement complet du document HTML.	Mise à jour ciblée des seuls nœuds du DOM ayant changé.
Expérience utilisateur	Coupures visuelles (page blanche entre deux écrans).	Fluidité proche d'une application native de bureau ou mobile.

1.2 La Problématique d'Échelle chez Facebook (2011)

Au début des années 2010, les applications web riches reposaient soit sur la manipulation directe du DOM via **jQuery**, soit sur des frameworks MVC/MVVM utilisant le **Two-Way Data Binding** (liaison de données bidirectionnelle), tels qu'AngularJS (1.x) ou Backbone.js.

En 2011, l'équipe d'ingénierie de Facebook se heurte à une impasse architecturale sur son interface web :

- L'application combine un fil d'actualité en temps réel, un compteur de notifications global et des fenêtres de messagerie instantanée (Chat).
- Dans un modèle à liaison bidirectionnelle, un événement réseau modifie un modèle de données, qui met à jour une vue, laquelle peut à son tour modifier un autre modèle par effet de bord.
- Cette architecture provoque des **mises à jour en cascade (cascading updates)** imprévisibles. Le symptôme le plus célèbre fut le « bug de la notification fantôme » : l'icône de messagerie affichait systématiquement des messages non lus alors que la fenêtre de conversation était vide ou déjà consultée.

1.3 Naissance et Évolution de React

Pour résoudre cette dette technique, **Jordan Walke**, ingénieur logiciel chez Facebook, conçoit un prototype interne baptisé **FaxJS**, inspiré des langages fonctionnels (Standard ML) et de la bibliothèque PHP interne XHP. Son idée directrice est radicale : **au lieu de muter le DOM morceau par morceau lorsqu'une donnée change, pourquoi ne pas régénérer**

virtuellement toute la vue comme une fonction pure de l'état, et laisser un moteur calculer les différences ?

Année	Jalon Historique	Impact Technique
2011	Déploiement interne chez Facebook	Mise en production de React sur le fil d'actualité (Newsfeed).
2012	Adoption par Instagram	Rachat d'Instagram ; leur équipe web adopte React et pousse à son ouverture.
2013	Open Source (JSConf US)	Publication officielle de React en mai 2013. Introduction de JSX.
2015	Écosystème Redux et React Native	Standardisation des flux de données unidirectionnels (Flux/Redux) et portage mobile.
2017	React 16 (Fiber)	Réécriture complète du moteur de réconciliation interne pour permettre le rendu asynchrone.
2019	React 16.8 (Hooks)	Révolution des Hooks (useState, useEffect) : les composants fonctionnels remplacent les classes.
2022+	React 18 et 19	Mode concurrentiel (Concurrent Rendering), transitions, Server Components et Actions.

---

2. Principes Architecturaux et Fonctionnement Interne

Contrairement à Angular qui est un framework complet et prescriptif, **React est une bibliothèque JavaScript dédiée à la construction d'interfaces utilisateur (couche Vue)**. Elle repose sur quatre piliers fondamentaux.

2.1 Approche Impérative vs Approche Déclarative

- **Programmation Impérative (Vanilla JS / jQuery) :** Vous décrivez **comment** modifier l'interface étape par étape. Vous devez connaître l'état actuel du DOM pour le faire transiter vers le nouvel état.
- **Programmation Déclarative (React) :** Vous décrivez **ce que** l'interface doit afficher pour un état donné. React se charge des opérations de transition dans le DOM.

Le principe fondamental de React s'exprime ainsi : **l'interface utilisateur est une fonction directe de l'état applicatif (UI = f(state))**. Dès que l'état change, l'interface se recalcule pour refléter fidèlement ce nouvel état.

Exemple en approche impérative (JavaScript natif / DOM API)

Le développeur pilote manuellement chaque mutation du nœud HTML :

```JavaScript
const btn = document.getElementById('auth-btn');
const statusText = document.getElementById('status-text');
let isAuthenticated = false;

btn.addEventListener('click', () => {
  isAuthenticated = !isAuthenticated;

  if (isAuthenticated) {
    btn.textContent = 'Se déconnecter';
    btn.classList.remove('btn-primary');
    btn.classList.add('btn-danger');
    statusText.textContent = 'Statut : Connecté';
  } else {
    btn.textContent = 'Se connecter';
    btn.classList.remove('btn-danger');
    btn.classList.add('btn-primary');
    statusText.textContent = 'Statut : Hors ligne';
  }
});
```

Exemple en approche déclarative (React)

Le développeur décrit uniquement la projection visuelle de l'état :

```JavaScript
import { useState } from 'react';

export function AuthStatus() {
```

```
  const [isAuthenticated, setIsAuthenticated] = useState(false);

  return (
    <div>
      <p>Statut : {isAuthenticated ? 'Connecté' : 'Hors ligne'}</p>
      <button
        className={isAuthenticated ? 'btn-danger' : 'btn-primary'}
        onClick={() => setIsAuthenticated(!isAuthenticated)}
      >
        {isAuthenticated ? 'Se déconnecter' : 'Se connecter'}
      </button>
    </div>
  );
}
```

2.2 Le Coût du DOM Réel et le Concept de DOM Virtuel

Le **DOM (Document Object Model)** est la représentation arborescente en mémoire des éléments HTML gérée par le navigateur. Manipuler un objet JavaScript en mémoire vive prend quelques nanosecondes, mais modifier le DOM réel est une opération coûteuse car elle déclenche le pipeline graphique du navigateur :

1. **Recalcul des styles (Recalculate Style) :** Évaluation des sélecteurs CSS impactés.
2. **Mise en page (Layout / Reflow) :** Calcul géométrique des positions et dimensions des boîtes dans le document.
3. **Peinture (Paint) :** Rastérisation des pixels sur les différents calques.
4. **Composition (Composite) :** Assemblage des calques par le processeur graphique (GPU).

Pour éviter des opérations de Reflow répétitives et non optimisées, React maintient en mémoire une représentation légère du DOM sous forme d'objets JavaScript simples : le **DOM Virtuel (Virtual DOM)**.

JavaScript

```
// Représentation en mémoire d'un nœud du Virtual DOM (ReactElement) :
// Un simple objet JavaScript léger, sans les centaines de propriétés lourdes d'un
nœud DOM natif
{
  type: 'button',
  props: {
    className: 'btn-primary',
    children: 'Se connecter'
  },
```

```
  key: null,
  ref: null
}
```

2.3 L'Algorithme de Réconciliation et React Fiber

Lorsqu'un état change dans l'application, React exécute un processus en trois étapes appelé **Réconciliation** :

1. **Phase de Rendu (Render Phase) :** React appelle les fonctions de composants impactées et génère un nouvel arbre de DOM Virtuel représentant le nouvel état.
2. **Phase de Comparaison (Diffing) :** React compare ce nouvel arbre virtuel avec l'arbre virtuel du rendu précédent pour isoler les différences.
3. **Phase d'Application (Commit Phase) :** React applique au DOM réel **uniquement la liste minimale de mutations nécessaires** en une seule passe groupée.

Optimisation algorithmique : du cubique au linéaire

En informatique théorique, calculer la distance d'édition minimale entre deux arbres quelconques possède une complexité cubique (en N au cube). Pour un arbre de 1 000 éléments, cela représenterait un milliard d'opérations de comparaison, ce qui figerait totalement le navigateur.

React ramène cette complexité à un temps linéaire (proportionnel au nombre d'éléments N) grâce à deux hypothèses heuristiques :

1. **Deux éléments de types différents produisent des arbres différents :** Si une balise `<div>` devient une balise `<section>`, React ne tente pas de recycler le nœud ; il détruit le sous-arbre entier et le reconstruit à neuf.
2. **L'identification explicite par clé (key) :** Sur les listes dynamiques, le développeur fournit un attribut `key` unique et stable permettant à React de repérer les déplacements, insertions ou suppressions d'éléments sans tout recréer.

Le moteur React Fiber

Depuis React 16, le moteur interne s'appelle **Fiber**. Il découpe le travail de rendu en unités élémentaires appelées fibres. Cela permet à React de :

- Mettre en pause un calcul de rendu lourd si l'utilisateur tape au clavier ou clique sur un bouton.
- Prioriser les mises à jour urgentes (animations, saisies utilisateur) par rapport aux mises à jour secondaires (affichage d'un tableau reçu du réseau) afin de maintenir un

taux de rafraîchissement fluide à 60 images par seconde (soit un budget d'environ 16 millisecondes par image).

2.4 Le Flux de Données Unidirectionnel (One-Way Data Flow)

Dans React, les données circulent strictement **du haut vers le bas** (du composant parent vers les composants enfants) via des paramètres appelés `props`.

- **Descente des données :** Le composant parent détient l'état (state) et le transmet à ses enfants via les `props`. Un composant enfant ne peut jamais modifier directement les `props` qu'il reçoit (elles sont en lecture seule).
- **Remontée des événements :** Pour qu'un enfant déclenche une modification de l'état de son parent, le parent lui transmet une fonction de rappel (callback) en tant que prop, que l'enfant appelle lors d'une interaction utilisateur.

Ce flux unidirectionnel garantit un **débogage déterministe** : si une donnée affichée à l'écran est erronée, il suffit de remonter la chaîne des parents pour trouver la source exacte de l'état.

---

3. L'Environnement d'Exécution : Node.js et npm

Bien que le code React final s'exécute dans le navigateur du client, son développement moderne nécessite un environnement local basé sur **Node.js**.

3.1 Pourquoi Node.js pour du Front-End ?

Node.js est un environnement d'exécution JavaScript côté système (hors navigateur), construit sur le moteur V8 de Chrome. En développement React, Node.js ne sert pas nécessairement de serveur de production, mais il fait tourner toute **l'usine logicielle de développement (Toolchain)** :

- Le serveur de développement local avec rechargement à chaud (Hot Module Replacement - HMR).
- Les transpileurs (transformation du JSX et du JavaScript moderne vers un JavaScript compatible avec tous les navigateurs).
- Les outils de qualité de code (ESLint, Prettier) et les moteurs de tests (Vitest, Jest).
- Les bundlers (Webpack, Vite, Rollup) qui assemblent, optimisent et réduisent le code pour la production.

3.2 Gestion des Paquets avec npm et package.json

npm (Node Package Manager) est le gestionnaire de paquets officiel fourni avec Node.js. Il remplit deux rôles :

1. Un registre mondial hébergeant les bibliothèques open source (react, react-dom, redux, etc.).
2. Un outil en ligne de commande (CLI) permettant d'installer ces bibliothèques et d'exécuter les scripts du projet.

Tout projet React comporte à sa racine un fichier manifeste `package.json` :

JSON

```
{
  "name": "dashboard-react-spa",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "eslint": "^9.9.0",
    "vite": "^5.4.1"
  }
}
```

Distinction fondamentale : dependencies vs devDependencies

- **dependencies (`npm install <nom_paquet>`) :** Bibliothèques **incluses dans le code final envoyé au navigateur** des utilisateurs (ex. react, react-dom, react-router-dom).
- **devDependencies (`npm install -D <nom_paquet>`) :** Outils utilisés **uniquement sur la machine du développeur ou le serveur d'intégration continue (CI/CD)** pour compiler, vérifier ou tester le code (ex. vite, eslint). Elles ne sont pas ajoutées au bundle final de production.

3.3 Le Versionnage Sémantique (SemVer) et package-lock.json

Les numéros de version dans `package.json` suivent la norme **SemVer** : `MAJEUR.MINEUR.CORRECTIF` (ex. `18.3.1`).

- **MAJEUR (18) :** Changements incompatibles avec les versions précédentes (Breaking Changes).
- **MINEUR (3) :** Ajout de fonctionnalités rétrocompatibles.
- **CORRECTIF (1) :** Correction de bugs ou de failles de sécurité rétrocompatibles.

Les préfixes dans `package.json` définissent la politique de mise à jour autorisée :

- `^18.3.1` (Caret) : Autorise les mises à jour mineures et correctives (de 18.3.1 inclus jusqu'à 19.0.0 exclu).
- `~18.3.1` (Tilde) : Autorise uniquement les correctifs (de 18.3.1 inclus jusqu'à 18.4.0 exclu).
- `18.3.1` (Exact) : Verrouille strictement cette version.

Le fichier `package-lock.json` fige l'arbre exact des versions téléchargées (avec leur empreinte de sécurité SHA-512). Il doit impérativement être versionné dans Git pour garantir que tous les développeurs de l'équipe travaillent avec des dépendances strictement identiques.

---

4. La Chaîne de Compilation : Webpack et Vite

Les navigateurs web ne comprennent nativement ni la syntaxe **JSX** propre à React, ni la résolution directe des imports depuis le dossier `node_modules` sans configuration spécifique. Il faut donc un **transpileur** et un **bundler**.

4.1 Le Rôle du Transpileur et du Bundler

1. **Transpilation (Babel / SWC / esbuild) :** Convertit le code source écrit par le développeur (JSX et syntaxes modernes) en instructions JavaScript standards compréhensibles par tous les navigateurs.
2. **Bundling (Webpack / Rollup) :** Parcourt l'arborescence des fichiers (imports et exports), résout les dépendances, intègre le CSS et les images, élimine le code mort (Tree-Shaking) et génère des fichiers optimisés pour la production.

4.2 Architecture et Configuration de Webpack

Webpack repose sur 5 concepts clés :

1. **entry :** Le point d'entrée de l'application (généralement `./src/index.jsx`) à partir duquel Webpack construit son graphe de dépendances.
2. **output :** L'emplacement et le nommage des fichiers générés après compilation (ex. `./dist/bundle.js`).
3. **loaders :** Permettent de pré-traiter les fichiers autres que JS/JSON (`babel-loader` pour le JSX, `css-loader` et `style-loader` pour le CSS).

4. **plugins :** Étendent les capacités de compilation (ex. `HtmlWebpackPlugin` pour injecter automatiquement les scripts dans le fichier HTML).
5. **mode :** Défini sur `'development'` (compilation rapide, Source Maps détaillées) ou `'production'` (minification maximale, Tree-Shaking).

Exemple d'un fichier webpack.config.js type pour React

JavaScript

```
const path = require('path');
const HtmlWebpackPlugin = require('html-webpack-plugin');

module.exports = {
  entry: './src/index.jsx',
  output: {
    path: path.resolve(__dirname, 'dist'),
    filename: 'bundle.[contenthash].js',
    clean: true,
  },
  mode: 'development',
  resolve: {
    extensions: ['.js', '.jsx'],
  },
  module: {
    rules: [
      {
        test: /\.(js|jsx)$/,
        exclude: /node_modules/,
        use: {
          loader: 'babel-loader',
          options: {
            presets: [
              '@babel/preset-env',
              ['@babel/preset-react', { runtime: 'automatic' }]
            ],
          },
        },
      },
      {
        test: /\.css$/,
        use: ['style-loader', 'css-loader'],
      },
    ],
  },
  plugins: [
    new HtmlWebpackPlugin({
      template: './public/index.html',
    }),
  ],
```

```
  devServer: {
    static: path.join(__dirname, 'public'),
    port: 3000,
    hot: true,
    historyApiFallback: true,
  },
};
```

4.3 L'Approche Moderne avec Vite

Sur des projets volumineux, Webpack présente une limite : il doit parcourir et compiler **l'intégralité du code de l'application** avant de pouvoir démarrer le serveur local.

Vite résout ce problème en adoptant une approche différente :

- **En développement :** Vite ne génère pas de bundle complet au démarrage. Il s'appuie sur le support natif des **ES Modules** dans les navigateurs modernes et pré-compile uniquement les dépendances tierces avec **esbuild** (écrit en Go, ultra-rapide). Le serveur démarre en quelques fractions de seconde, quelle que soit la taille de l'application.
- **En production :** Vite utilise **Rollup** pour produire un bundle final optimisé, découpé et minifié.

Critère	Webpack	Vite
Principe en développement	Bundle complet compilé en mémoire avant de servir	Serveur ES Modules natif à la demande
Moteur de transpilation	babel-loader (JS) ou swc-loader	esbuild (Go) / SWC (Rust)
Temps de démarrage à froid	Proportionnel à la taille globale du projet	Quasi instantané (indépendant du volume de code source)
Configuration requise	Verbeuse (fichiers de configuration séparés)	Minimale (vite.config.js prêt à l'emploi)
Usage typique	Projets existants, architectures Micro-Frontends	Nouveaux projets SPA React, rapidité d'itération

---

5. Anatomie et Cycle d'Amorçage d'une Application React

Une application React repose sur la séparation entre deux bibliothèques fondamentales :

1. **react :** Le cœur agnostique. Contient la logique des composants, des états (useState), des effets (useEffect) et du DOM Virtuel.
2. **react-dom :** Le moteur de rendu spécifique au navigateur web, chargé de traduire le DOM Virtuel en opérations sur le DOM HTML réel.

5.1 Organisation Standard des Fichiers

Fichier ou Dossier	Rôle dans l'architecture du projet
`node_modules/`	Dossier contenant toutes les dépendances installées (exclu de Git via .gitignore).
`public/`	Fichiers statiques servis tels quels sans compilation (favicon, images brutes).
`src/`	Code source de l'application compilé par le bundler.
`src/assets/`	Ressources multimédias importées dans les composants.
`src/components/`	Composants d'interface réutilisables.
`src/App.jsx`	Composant racine de l'application assemblant les vues.
`src/main.jsx`	Point d'entrée technique JavaScript assurant le montage de React dans le DOM.
`index.html`	Document HTML unique de la SPA contenant la balise `<div id="root"></div>`.
`package.json`	Manifeste listant les scripts, les dépendances et les métadonnées du projet.
`vite.config.js`	Fichier de configuration du bundler et de ses plugins.

5.2 Le Cycle d'Amorçage (Bootstrapping)

Le démarrage d'une Single Page Application React suit un enchaînement précis en 3 fichiers :

Étape 1 : Le conteneur HTML (index.html)

Le fichier HTML contient un unique élément `<div id="root"></div>` vide. C'est le point d'ancrage dans lequel React va injecter toute l'interface.

HTML

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Application React SPA</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Étape 2 : L'initialisation de la racine React (src/main.jsx)

Depuis React 18, on utilise l'API `createRoot` issue de `react-dom/client` pour activer le moteur concurrentiel :

JavaScript

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';
import './index.css';

// 1. Ciblage du nœud DOM hôte
const container = document.getElementById('root');

// 2. Création de la racine React
const root = createRoot(container);

// 3. Premier rendu de l'arbre de composants
root.render(
  <StrictMode>
    <App />
  </StrictMode>
```

```
);
```

Étape 3 : Le composant racine (src/App.jsx)

Le composant `App` est une fonction JavaScript qui retourne la structure visuelle initiale :

```JavaScript
export default function App() {
  return (
    <main className="app-container">
      <h1>Plateforme Applicative React</h1>
      <p>Environnement initialisé avec succès.</p>
    </main>
  );
}
```

5.3 Le Rôle de StrictMode

Le composant englobant `<StrictMode>` présent dans `main.jsx` est un outil de diagnostic **actif uniquement en mode développement** (sans impact en production) :

- Il **exécute deux fois** le corps des fonctions de composants et les effets lors du montage initial.
- **Objectif :** Vérifier que les composants sont des **fonctions pures** et que les effets nettoient correctement leurs ressources (timers, abonnements). Si doubler l'exécution d'un composant fausse l'affichage, `<StrictMode>` signale l'anomalie dès la phase de développement.
- Il avertit dans la console lors de l'utilisation d'API dépréciées.

5.4 Outil de Diagnostic : React Developer Tools

Pour inspecter une application React, l'extension navigateur **React Developer Tools** ajoute deux onglets indispensables aux outils de développement :

1. **Components :** Visualise l'arbre hiérarchique réel des composants React, permettant d'inspecter et modifier en direct leurs props et leur état interne.
2. **Profiler :** Permet d'enregistrer les interactions, de visualiser quels composants se re-rendent, d'identifier les causes de re-rendu et de mesurer le temps d'exécution de chaque composant.

---

6. Synthèse

1. **Paradigme Déclaratif :** On ne manipule plus manuellement les nœuds du DOM avec `document.querySelector`. On modifie l'état interne, et React synchronise la vue selon la logique `UI = f(state)`.
2. **Virtual DOM et Réconciliation :** React maintient un arbre virtuel léger en mémoire et applique un algorithme de comparaison linéaire pour minimiser les opérations réelles sur le DOM.
3. **Flux Unidirectionnel :** Les données descendent des composants parents vers les composants enfants via les `props`, facilitant la traçabilité et le débogage.
4. **Outillage Moderne :** Node.js sert de socle pour l'outillage de développement. Vite s'appuie sur les ES Modules natifs pour un confort de développement instantané, tandis que Webpack offre une configuration granulaire historique.